Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



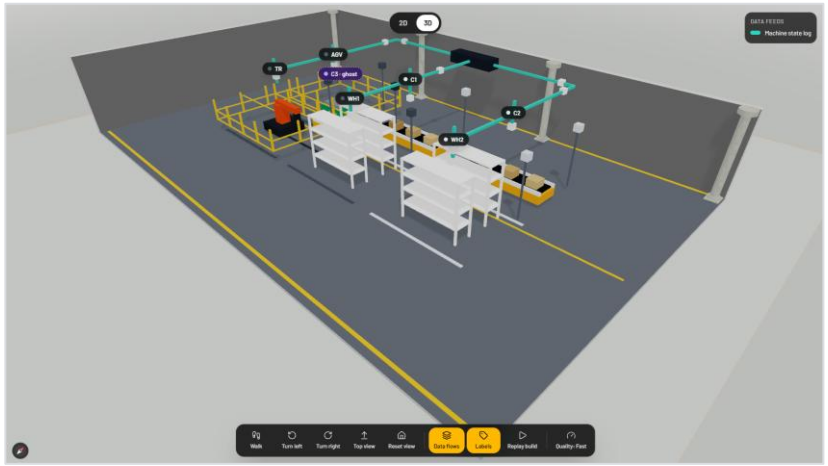
Team Details

a. Team name: Lanterns
b. Team leader name: Shreyash Sharma. Team member: Kritika Tiwari
c. Problem Statement: Manufacturing: Intelligent Operations & Industrial Efficiency. Plants run on scattered knowledge: manuals, orders, machine logs. When a station fails, nobody can quickly tell why, or whether a fix is safe. We built an agentic platform around a Symbiotic Compiler: it compiles a plant from its own documents and data, finds the cause, and acts only inside limits the owner sets.

Brief about the idea

An agentic platform for manufacturing, built around a Symbiotic Compiler. It compiles a plant's own documents and data into one checked knowledge graph, finds why a station failed, and acts only inside limits the owner sets.

- **Nine agents, one gateway.** Four read the documents, connect the systems and build the model. Five run it: Diagnosis, Verifier, Planner, Warden and Sentinel.
- **Evidence before action.** Every fact keeps the exact quote it came from. Instructions hidden in a file are removed, never obeyed. The Verifier recomputes every number a diagnosis cites.
- **Autonomy is a dial.** Warden scores each action 0 to 100. Below 40: refused. From 40 to 79: a person approves. From 80: it runs alone. The owner can change 40 and 80. A hard limit from the documents is never crossed.
- **Built on Google Cloud.** Gemini on Vertex AI in every agent, Cloud Run, Firebase sign-in.



9
agents

~30 s
failed station to scored plan, median of 7 local runs

17
system types read from documents

Why it matters

The problem

- **Unplanned downtime.** A station fails and the line waits.
- Nobody can quickly tell why it failed, or whether a fix is safe.
- The knowledge sits in manuals, orders, machine logs and a few people's heads.

The cost

- Every hour a line is down is output the plant does not get back.
- **Unplanned downtime costs the world's 500 largest companies about $1.4 trillion a year, 11% of their revenue (Siemens, The True Cost of Downtime 2024).**

Who benefits, and how it scales

- Small and mid-size plants that have no reliability team.
- Onboarding works for any plant whose systems can be connected.
- The owner keeps the authority. Warden runs an action alone only when its score reaches the owner's threshold (80 by default).

Opportunities

How is it different from other ideas?

The Symbiotic Compiler

- It reads the documents and data of your plant.
- It builds a knowledge graph of stations, systems and flows. Each fact has its quote.
- A person confirms the graph. The agents use it. This is the symbiosis.

How will it solve the problem?

From a fault to a fix

- A station fails. Diagnosis finds the cause in logs, databases and APIs.
- The Verifier checks the numbers. The Planner proposes a fix with exact settings.
- Warden scores the fix. The fix runs alone, a person approves it, or it is refused.

USP of the solution

Four things that work together

- Auto-remediation. It finds the cause and fixes it. Warden keeps the owner in control.
- Built from your documents. No template. Each fact has its quote.
- It asks for each system by name. It knows 17 system types. It only reads.
- It checks itself. The Verifier recomputes every number before action.

Maintenance copilots, such as MaintainX CoPilot and Siemens Industrial Copilot, answer a technician's questions. Tolerance learns the plant from its own documents. It puts every action through a gate the owner controls.

The agent team and the services around it



The agent team: owner, Guide, onboarding and operations agents, shared services, policy gateway and machines.

How it fits together

- **Cloud Run:** one container with the web app and the API.
- **Foreman (our agent runner):** runs the nine agents and records every step.
- **Gemini on Vertex AI:** writes the words, reads answers, picks tools.
- **Cedar (open-source policy language):** default deny. Nothing reaches a machine except through it.
- **MCP hub:** built-in and your own servers. Only read-only tools run.
- **Plant knowledge graph:** stations, systems and flows. Each edge keeps its evidence, and counts only if seen in runtime data.
- **Data lake:** DuckDB, kept current by connectors.

List of features offered by the solution

Guide onboarding
Asks the plant name, reads the documents, settles conflicts, draws the plant.

Document Reader
Facts with exact quotes. Hidden instructions are removed first.

Live connectors (17 types)
Tested live: PostgreSQL, MQTT, MongoDB. Built, not yet tested live: MySQL, SQL Server, Oracle, Kafka, OPC UA.

Your own MCP servers
They join the same hub. Only read-only tools can run.

Floor in 2D and 3D
A 2D plan and 3D view, with every data feed joined to its stations.

Layout editor
Move, add and delete stations. Draw how parts flow.

Operations
Diagnosis, Verifier, Planner and Warden in one run, step by step.

Warden gate
Score 0 to 100. Below 40 refused, from 80 runs alone. Hard limits. A backtest.

Sentinel
Plain rules on data, services and certificates. Maintenance windows.

Playbooks
Draw what should happen when a station fails.

Roles and approval
Cedar decides who may approve. Telegram buttons, Slack, Teams, Discord or a webhook.

Private by design
Firebase sign-in, a private plant per user, encrypted passwords, usage caps.

Process flow diagram or Use-case diagram

Set up once: the Symbiotic Compiler builds the plant

Give the plant name and documents → Connect the systems the documents name → The knowledge graph is compiled, each fact with its quote → The owner confirms what was understood → The floor goes live in 2D and 3D → The owner sets the score thresholds (40 and 80)

When a station fails

Diagnosis
finds the cause with read-only tools

→ **Verifier**
recomputes every number it cites

→ **Planner**
proposes the fix with exact settings

→ **Warden**
hard limits, then a score from 0 to 100

→
- **Blocked or refused:** does not run
- **A person approves:** in the app or chat
- **Runs by itself:** score 80 or more

Then: the Action Broker applies only what the gate allows, in a simulator. Every model call, tool call and decision is recorded.
Who uses it: the owner sets it up and the limits, an operator reads the floor, a supervisor approves the middle zone.

One incident, start to finish

1 2 3 4 5 6

Fault

A station or node in the plant keeps stopping. The Operations run starts.

Diagnosis

Finds the cause with read-only tools on the plant's data: station logs, the MES database, APIs. Result: repeated stops and a 0.3 chance of another in 4 hours. Confidence 70%.

Verifier

Recomputes the numbers from the same data. They match, so the diagnosis stands. No hard limit is broken.

Planner

Proposes three actions: inspect the station, schedule maintenance, cap its speed at 70%.

Warden

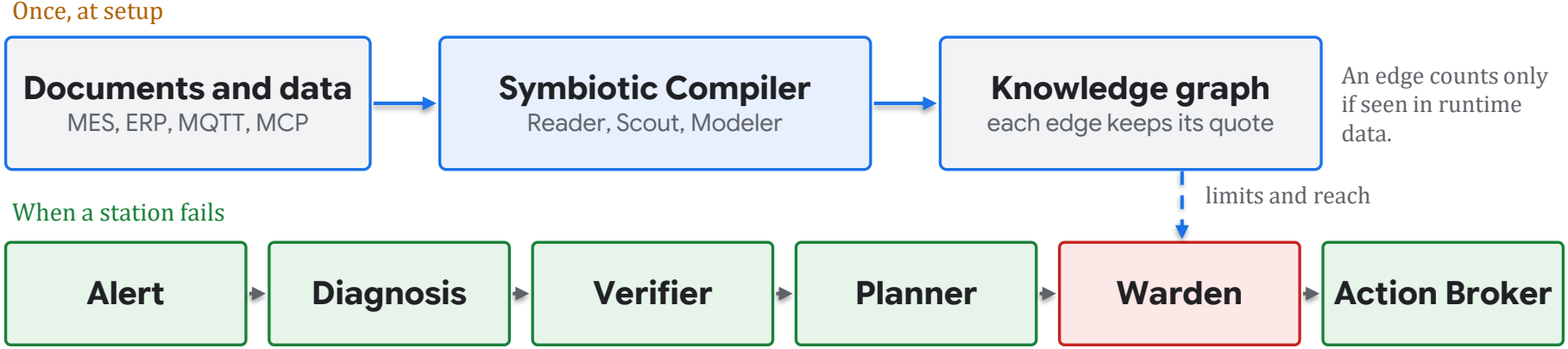
Scores each action 0 to 100 from six factors: undo, reach, limits, evidence, history, plant state. Inspect (88) and maintenance (84) reach the threshold of 80 and run alone. The speed cap (74) is between 40 and 80, so a person decides.

Approval

The plant owner approves in the app. The change runs in the simulator, about 30 s after the alert. Every step is in the log.

Taken from a recorded run. Every step is in the audit log.

Architecture diagram of the proposed solution



$$F(c) = (1/K)\,\Sigma_p\, s_p(c) \,/\, \max_k s_p(k)$$

Metric, trace and log probes fused into one ranking of causes.

Runs on Cloud Run, Vertex AI and Firebase.

Alone: S ≥ 80	Approve: 40 to 79	Refused: S < 40

$$S = \Sigma_i\, 100 \cdot w_i \cdot v_i$$

Six factors. A broken hard limit blocks at any score.

+0.08 Avg@5 over BARO, public software benchmark (RCAEval)

32% of 465 benchmark incidents (RCAEval) run alone

~30 s failed station to scored plan, median of 7 local runs

Technologies to be used in the solution

Google Cloud

- Gemini on Vertex AI (Gemini 2.5 Flash and the flash-lite family)
- Cloud Run, one always-on instance
- Firebase Authentication (Google, email, guest)
- Cloud Build and Artifact Registry

Agents and safety

- Foreman agent runner, Gemini function calling
- Cedar policy language, default deny
- Model Context Protocol (hub and client)
- Rule-driven Warden and Sentinel

Data and connectors

- DuckDB lake, dlt loaders
- SQLAlchemy: PostgreSQL, MySQL, SQL Server, Oracle
- pymongo, MQTT, Kafka, OPC UA (asyncua)
- Encrypted vault for passwords

Web

- Next.js 16 and React 19
- 2D and 3D floor-plan viewer
- three.js, React Flow
- FastAPI backend

Gemini is called through one gateway: a daily cap, model rotation when a model is busy, recorded answers as a fallback, and a visible notice when Gemini is under load.

Estimated implementation cost (optional)

Item	What it covers	Pilot, 1 plant	Production, 10 plants
Cloud Run	Web app and API. Today: 1 instance. Production: 3 instances in different zones.	~$120	~$380
Gemini on Vertex AI	About $0.02 for each incident. 20 incidents a day today, 200 in production, plus onboarding.	~$11	~$140
Database	Cloud SQL with high availability: 4 vCPU, 16 GB, 200 GB. Today: files and DuckDB in the container.	$0	~$500
Network and security	Load balancer, Cloud Armor, and VPN links to the plant networks.	$0	~$190
Logs, storage, build	Audit logs, monitoring, backups, secrets, and one build for each deploy.	~$2	~$85
One month	**Production is about $130 for each plant.**	**~$135**	**~$1,300**

Estimate at list prices, in US dollars. It leaves out people, a support plan and licences. Gemini: about 3,000 input and 500 output tokens for each call.

How the product performs

~30 s
failed station to scored plan, median of 7 local runs

~9
Gemini calls per incident, one measured run

12 / 4 / 0
actions run alone / sent for approval / blocked (16 actions, 7 demo incidents)

0.2 s
a read-only MCP tool call, measured locally

Checks that pass today

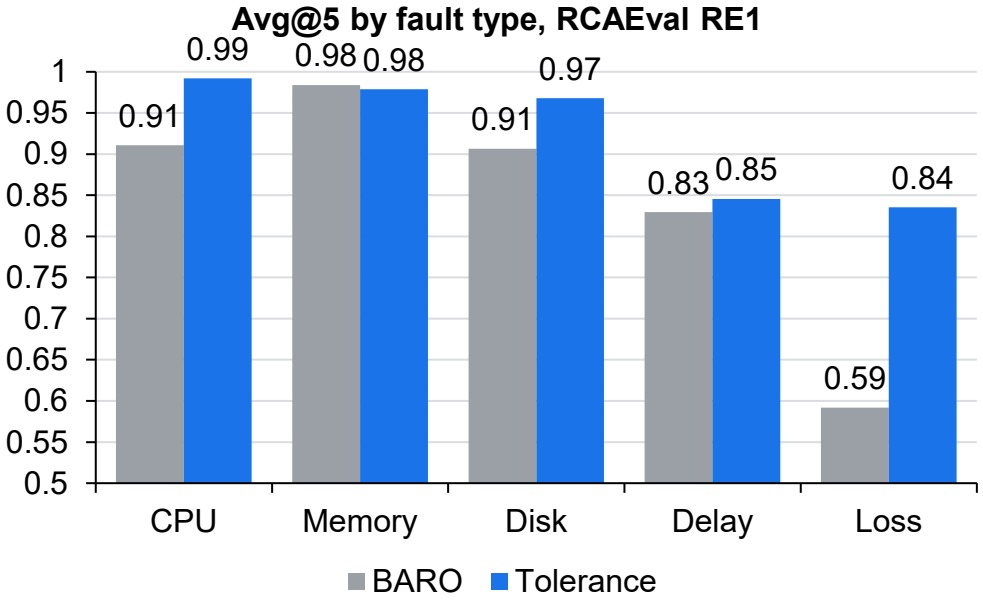
- 12 automated suites: policy, agents, Sentinel, probes, connectors, vault, MCP.
- PostgreSQL, MQTT and MongoDB connected live, loading only what is new.
- A remote MCP server: a read-only tool runs, an acting tool is denied.
- Not yet tested live: MySQL, SQL Server, Oracle, Kafka, OPC UA.

Research result

Root-cause ranking on a public software benchmark (RCAEval RE1, 298 cases): +0.08 Avg@5 over BARO, 95% interval +0.05 to +0.12. No Gemini in this test.

Figures come from the running prototype on one computer. Gemini's speed varies with load; the app then shows a notice.

Benchmark: finding the root cause



+0.08

Avg@5 over BARO, 95% interval +0.05 to +0.12

298

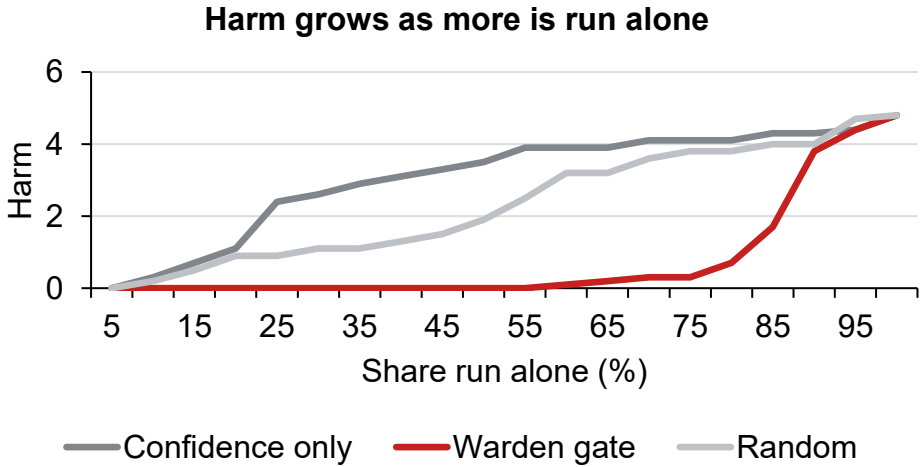
cases, 3 systems, 4 fault types

- Public software benchmark (RCAEval), not factory data.
- BARO is the published baseline, run unmodified.
- Largest gain on packet loss: 0.59 to 0.84.
- No Gemini in this test.

Per-system detail is in the appendix. On Train Ticket the gain is +0.19 against our BARO run (0.66); the BARO paper reports 0.79 for this system.

Warden keeps irreversible actions with a person

At the default thresholds (40 and 80), 149 of 465 benchmark incidents (32%) run alone; the rest wait for a person.



465 incidents from a public software benchmark (RCAEval)

Policy	Run alone	Wrong	Harm
Run all alone	465	116	4.8
Random 149	149	33	1.1
Confidence only	149	19	2.6
Warden gate	149	29	0.0

Warden lets more wrong diagnoses run alone (29) than confidence alone (19), but only actions that can be undone, so harm stays at zero. Harm is weighted by our action table's reversibility values, so low harm partly follows from the gate's design. The chart shows how much autonomy that costs.

Additional Details/Future Development (if any)

A pilot with a real plant	Replace the replayed lab log and software incidents with a customer's own history.
Tune Gemma for Sentinel	Gemini sorts short texts into labels today. We plan to fine-tune Gemma for that one classification job, with plain rules still judging the result.
Reach private plant networks	Probes or a tunnel so a cloud copy can read systems inside a factory network.
Write actions behind Warden	Let remote MCP tools act, only through the gate and with a person's approval.
More live checks	Test Kafka, Oracle, SQL Server, MySQL and OPC UA against live servers. Add roles for approvers and many plants per owner.

Provide links to your:

- GitHub Public Repository: https://github.com/shreyash-sharma/tolerance
- Demo Video Link (3 Minutes): **[FILL: demo video URL]**
- Final Product Link: https://tolerance-749288317336.asia-southeast1.run.app

How to try it

1

Open the final product link.

2

Sign in with your Google ID. Or Register, or Try it as a guest. A guest gets a private plant with usage caps.

3

Give a plant name and drop documents, or load the made-up plant that has MES, MongoDB, ERP and IoT data.

Google Cloud H2S

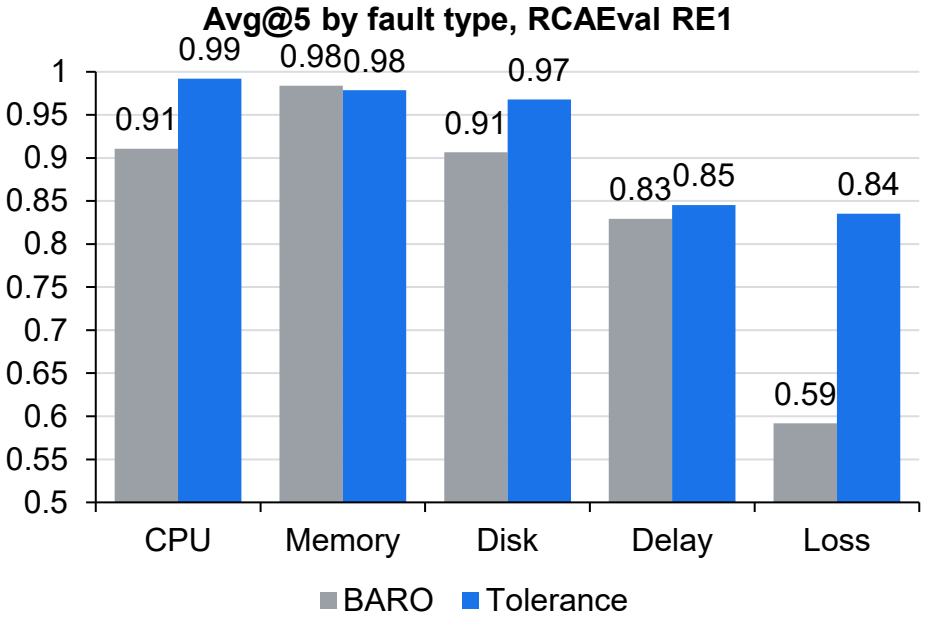
AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you



Appendix: Root-cause ranking detail (RCAEval RE1)



Difference from BARO, Avg@5 (95% interval)

Subset	n	Tolerance minus BARO
all	298	+0.08 [+0.05, +0.12]
Train Ticket	100	+0.19 [+0.10, +0.27]
Online Boutique	98	+0.04 [-0.01, +0.10]
Sock Shop	100	+0.02 [-0.01, +0.05]

- BARO is already at 0.96 on Sock Shop, so there is little room.
- Packet loss gains most by fault: 0.59 to 0.84.
- Pooled rows use the four faults the BARO paper reports.

Pooled: +0.08 Avg@5 over BARO (interval +0.05 to +0.12). Train Ticket: +0.19 against our BARO run (0.66); the BARO paper reports 0.79 for this system.

Appendix: What the results cover, and the next tests

What the results cover

- Software incidents on RCAEval. A factory pilot is the next step.
- The probes were developed on these cases, so RE2 is the held-out test.
- Harm uses the action table's reversibility values, not measured damage.
- The backtest uses the metrics-first diagnosis, not Gemini.
- Our run of BARO on Train Ticket is 0.65; the paper reports 0.79.

What we run next

- RCAEval RE2 (270 cases), the paper's main table, as a held-out test.
- The Gemini agent on RE3, where log text should matter, inside the credit budget.
- The other baselines (CIRCA, RCD, CausalRCA) once their dependencies fit.
- A customer's own incident history replayed through Warden.

Reproduce it: python -m bench.research reads the per-case rankings in bench/results and the 465 benchmark incidents (RCAEval) in seed. No downloads, runs in seconds.

Appendix: from industrial RCA research to evidence-first agents

Earlier research

A study of root-cause recommendation using structured incident metadata and machine-learning methods.

8,128 historical EV incident records studied.

Structured priors proved competitive with more complex models.

Status: accepted at ICCIS 2026 (International Conference on Communication and Intelligent Systems, BITS Pilani, Goa) for Springer's Lecture Notes in Networks and Systems. Awaiting print.

Tolerance's design

An independent agentic system that connects operational knowledge to evidence-backed diagnosis and controlled remediation.

- Structured evidence first
- Gemini for investigation
- Deterministic safety checks

Research insight. Not every industrial fault needs a complex neural network. Structured knowledge can provide strong diagnostic signals. Agentic reasoning can then investigate missing evidence and support controlled decisions.

Prior research and the Tolerance prototype are separate implementations. The historical dataset and proprietary assets are not part of the submitted system.

Appendix: how the numbers work

Warden score

$S = \Sigma_i\ 100 \cdot w_i \cdot v_i$

$w = 0.25, 0.20, 0.15, 0.25, 0.10, 0.05$

Six factors, each from 0 to 1: undo, reach, margin to limits, evidence, history, plant state.

Decision

blocked if a hard limit breaks

refused if $S < 40$, else approve if $S < 80$

Otherwise the action runs alone. The owner can change 40 and 80.

Factors

$v(\text{reach}) = 1 - r \cdot \min(1, n/4)$

$v(\text{history}) = 0.5 + 0.5\ (a - j)\ /\ (a + j + 2)$

n = stations downstream, a and j = past approvals and rejections.

Metric-shift probe

$\text{score} = \ln(1 + \min(d/\sigma, 60)) + d/\text{range}$

$\sigma = \max(1.4826\ \text{MAD}, 0.5\ \text{IQR}, 0.02\ |\text{med}|, \ldots)$

d = change of the median after the alert. The robust scale stops noise from winning.

Fusion and ranking

$F(c) = (1/K)\ \Sigma_p\ s_p(c)\ /\ \max_k\ s_p(k)$

$\text{Avg@5} = \tfrac{1}{5}\ \Sigma_{k=1..5}\ \text{AC@k}$

AC@k = share of cases with the true cause in the top k. Intervals: paired bootstrap.

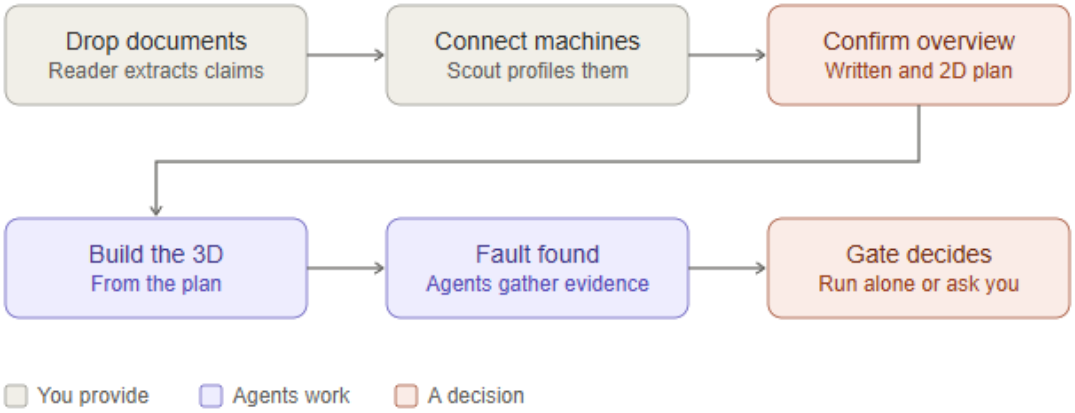
Harm and learning

$H = (1 - y)(1 - \text{rev})$

$\text{reward} = 1[\text{correct}] - \lambda \cdot \text{tool calls}$

A wrong action costs more when it is hard to undo. A bandit learns which evidence to read first.

Appendix: the journey, from documents to a decision



Boxes in grey are what you provide, in blue what the agents do, in red a decision.

Six steps

- You provide the documents and name the systems that hold the data.
- The agents read, connect and compile. You confirm what was understood.
- The plant is built from the checked facts, in 2D and 3D.
- When a fault is found, the agents gather evidence. The gate decides: it runs alone, or it asks you.

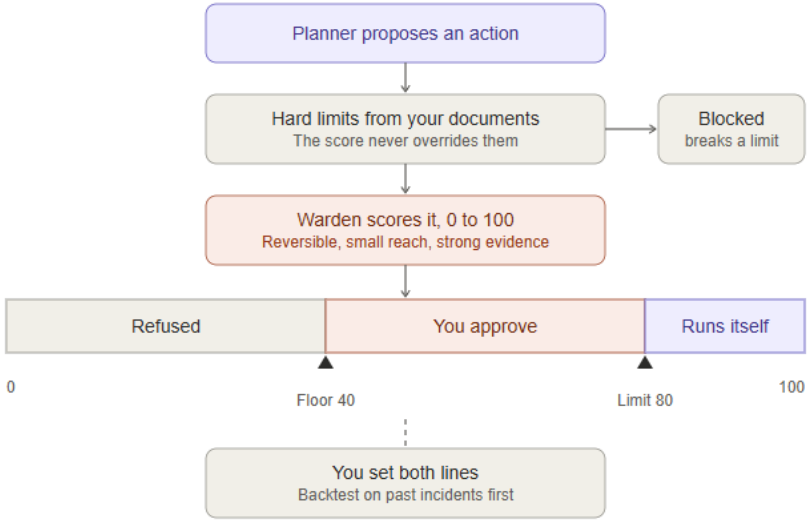
Appendix: roles and people, decided by Cedar

Permission	Owner	Supervisor	Operator	Viewer
See the floor and incidents	✓	✓	✓	✓
Run checks and diagnoses	✓	✓	✓	–
Approve or reject a fix	✓	✓	–	–
Change the score thresholds	✓	–	–	–
Connect systems and probes	✓	–	–	–
Manage alert channels	✓	✓	–	–
Add people and change roles	✓	–	–	–

How it works

- One policy engine for agents and for people: Cedar, default deny.
- The owner adds people and makes new roles on the Team page.
- Every request is checked on the server, not only hidden on the screen.
- A tap on Approve in Telegram follows the same rules. Each person links their own Telegram account.
- The audit log shows who approved, their role and the channel. A refusal is recorded too.

Appendix: the safety gate, limits first and then the score



Why this order

- A hard limit from your documents comes first. A score never overrides it.
- Warden then scores the action from 0 to 100 on six factors: can it be undone, how much it touches, distance from the limits, strength of the evidence, what people approved before, and the state of the plant.
- You set two score thresholds: 40 and 80 by default. Below 40 it is refused. From 40 to 79 you approve. From 80 it runs by itself.
- You see what the thresholds would have done on 465 incidents from a public software benchmark (RCAEval) before you trust them.

Appendix: the onboarding team

Guide

Model and rules · Gemini

Use case: A plant's documents disagree and have gaps. Only the owner can settle them.

Solution: Runs the Document Reader, Connector Scout and Plant Modeler, then asks the owner one question at a time. Gemini writes the words and reads free-text answers. Code checks each answer before it changes anything.

Outcomes: Settled conflicts · Confirmed limits · What the plant makes · A confirmed overview · A drawn layout · A tour of the app

Must never: Apply an answer that does not match an open question; Write to the plant; Obey instructions found inside documents

Document Reader

Model and rules · Gemini

Use case: Manuals, work instructions and safety sheets hold the limits and the structure of the plant.

Solution: Gemini extracts claims; code checks every quote against the page; instructions hidden in documents are removed first.

Outcomes: Claims with quotes · Conflicts between documents · Missing facts · Hidden instructions flagged

Must never: Obey instructions found inside documents; Write to the plant

Connector Scout

Model-driven · Gemini

Use case: A plant has ERP, MES, PLC and file data. Someone must say which is which.

Solution: MCP servers list sources and tables. Gemini names the role of each table from its columns.

Outcomes: Role of each table · Which sources are live · What data is missing

Must never: Write to the plant; Read rows outside SELECT

Plant Modeler

Model and rules · Gemini

Use case: The 3D model must come from facts a person has checked.

Solution: Stations from data, material flow from the documents, limits from the documents. Gemini writes the overview and each sentence cites its source.

Outcomes: Written overview · 2D plan · Limits list · Confirmation gate

Must never: Build the 3D before a person confirms

Appendix: the operations team

Diagnosis

Model-driven · Gemini

Use case: A station stopped. Someone must say why, fast.

Solution: Gemini picks MCP tools, reads the results and submits a cause with evidence. Cedar allows each call.

Outcomes: Cause · Evidence per claim · Tool trace

Must never: Write to the plant; Call a tool Cedar denies

Verifier

Model and rules · Gemini

Use case: A model can sound sure and be wrong.

Solution: Rule checks recompute every cited number from the tool results. Gemini then reviews the argument. A hard check cannot be talked away.

Outcomes: Supported or not · Issues list · Evidence score

Must never: Approve a claim whose numbers do not match

Remediation Planner

Model-driven · Gemini

Use case: After a cause is known, someone must choose a fix that respects the plant's limits.

Solution: Gemini proposes actions from the diagnosis and the limits. Warden scores each. Nothing is applied here.

Outcomes: Action list · Exact settings · Why this action

Must never: Write to the plant; Propose a setting above a hard limit as the first choice

Warden

Rule-driven · Gemini

Use case: Some fixes are small and some can stop a line. The user decides how far the system may go alone.

Solution: Six factors give a score from 0 to 100. The user sets two score thresholds, 40 and 80 by default. Cedar enforces them at the Action Broker.

Outcomes: Score with factors · Auto, approve or refuse · Backtest on past incidents

Must never: Let a high score override a hard limit

Appendix: plain rules, playbooks, channels, layout editor

Sentinel

Model and rules · Gemini

Use case: A plant's IT side fails quietly: a feed stops, a column changes, a service is slow. A model should not be the judge of that.

Solution: Plain rules check a field against a value on your data tables, a web address, a port, a service or a certificate. A schedule runs them, maintenance windows keep planned work quiet, and an impact map shows which stations lose data. A rule that fails or recovers tells your channels.

Outcomes: Pass or fail for each rule · The stations that lose data · A message when a rule fails or recovers

Must never: Write to the plant; Let a model say pass or fail; Reach a private network address

Playbooks

Rule-driven

Use case: Each plant reacts to a failure in its own way.

Solution: The owner draws a trigger, checks, a decision and actions on a canvas. Code walks it. Warden scores any action that changes the plant.

Outcomes: A trace of every block · Actions scored by Warden

Must never: Skip Warden for an action that changes the plant

Channels

Rule-driven

Use case: A person who is not in the app must still know when to approve.

Solution: Warden messages Slack, Teams, Discord, Telegram or a webhook. Addresses stay on the server. Private addresses are refused.

Outcomes: Approval requests · Blocked actions · Failing rules

Must never: Send to a private network address; Show an address back in the app

Layout editor

Rule-driven

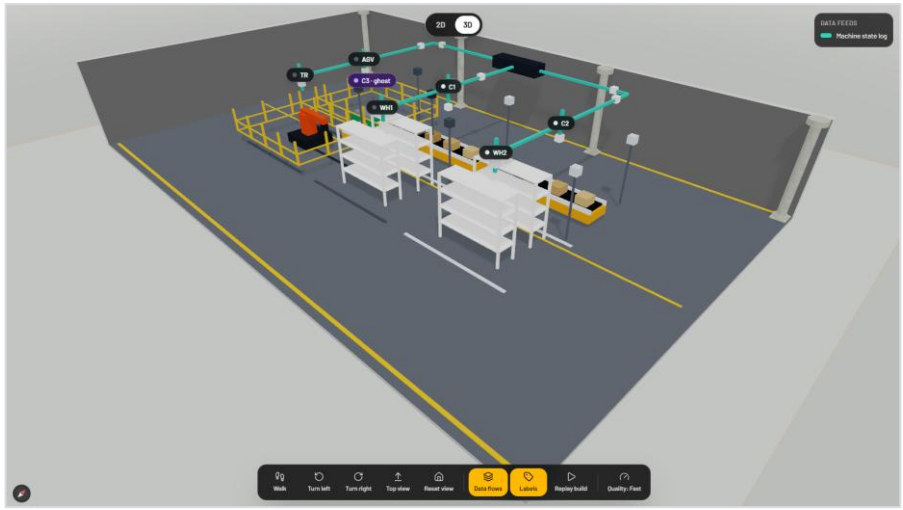
Use case: The drawn floor is never exactly right.

Solution: A node canvas next to the live floor: move, add and delete stations, rename them, and draw how parts flow. Every change is saved at once and the plan and 3D view follow.

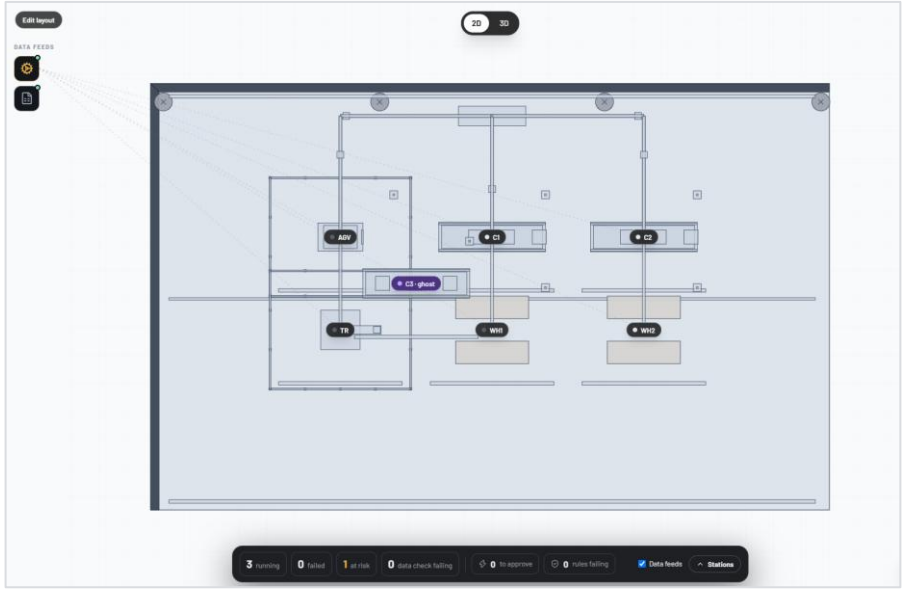
Outcomes: Moved, added or deleted stations · Part flows between stations

Must never: Change data or limits

Snapshots of the prototype



The same plant in 3D, with its data flows



The demo plant in 2D: data feeds, name tags and the status panel